



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET



Obrada transakcija i ACID izolacija u PostgreSQL-u

Seminarski rad
Studijski program: Elektrotehnika i računarstvo
Modul: Bezbednost računarskih sistema
Predmet: Sistemi za upravljanje bazama podataka

Student:

Katarina Adamović, br. ind. 2173

Mentor:

Prof. dr Aleksandar Stanimirović

Niš, maj 2026.

SADRŽAJ:

1. Uvod	4
2. Transakcije u relacionim bazama podataka	4
2.1. Definicija i osnovna svojstva	4
2.2. Životni ciklus transakcije	5
2.3. Transakcije u PostgreSQL-u - osnovna sintaksa	5
3. ACID svojstva	5
3.1. Atomičnost (Atomicity)	6
3.2. Konzistentnost (Consistency)	6
3.3. Izolacija (Isolation)	6
3.4. Trajnost (Durability)	6
4. Fenomeni pri konkurentnom izvršavanju	7
4.1. Prljavo čitanje (Dirty Read)	7
4.2. Neponovljivo čitanje (Non-repeatable Read)	7
4.3. Fantomsko čitanje (Phantom Read)	7
4.4. Serializaciona anomalija	8
5. Nivoi izolacije u PostgreSQL-u	8
5.1. Read Uncommitted	8
5.2. Read Committed	8
5.3. Repeatable Read	9
5.4. Serializable	9
6. MVCC - Kontrola konkurentnog pristupa zasnovana na verzijama	10
6.1. Principi rada MVCC-a	10
6.2. Vidljivost verzija u PostgreSQL-u	11
6.3. Vacuum i čišćenje starih verzija	11
7. Zaključavanje i deadlock	12
7.1. Vrste zaključavanja	12
7.2. Mrtva petlja (Deadlock)	13
7.3. Prevencija i rešavanje deadlock-a	13
8. Praktični primeri	14
8.1 Commit i Rollback:	16
8.2 Delimično poništavanje pomoću SAVEPOINT mehanizma	17
8.3 Konzistentnost (CHECK constraint):	18
8.4 Foreign Key:	19
8.5 Dirty Read (Izolacija):	20
8.6 Non-Repeatable Read:	22
8.7 Repeatable Read:	24
8.8 Phantom Read:	26
8.8 Repeatable Read sprečava Phantom Read:	28
8.9 SELECT FOR UPDATE:	29
8.10 NOWAIT:	31

8.11 Deadlock:.....	32
8.12 Dijagnostika sa pg_locks:	34
9. Zaključak	36
10. Literatura.....	37

1. Uvod

U savremenom svetu, baze podataka predstavljaju temelj gotovo svakog informacionog sistema — od bankarskih aplikacija i e-commerce platformi do medicinskih sistema i društvenih mreža. Svakodnevno se izvršavaju milijarde operacija nad podacima, a ključni izazov je osigurati da sve te operacije budu konzistentne, ispravne i pouzdane, čak i kada se odvijaju istovremeno.

Da bismo ilustrovali ovaj izazov, zamislimo jednostavan primer: korisnik želi da prenese novac sa jednog računa na drugi. Ova operacija uključuje dva koraka — smanjenje iznosa na jednom računu i povećanje na drugom. Šta se događa ako sistem otkaže između ta dva koraka? Novac bi bio skinut sa jednog računa, ali nikad dodat na drugi — nestao bi bez traga. Upravo ovakve scenarije rešavaju transakcije.

PostgreSQL je jedan od najnaprednijih sistema otvorenog koda za upravljanje relacionim bazama podataka. Poznat je po svojoj robusnosti, proširivosti i strogo poštovanju ACID principa — skupa svojstava koja garantuju pouzdanost transakcija u svakom okruženju. Zahvaljujući sofisticiranim mehanizmima za upravljanje konkurentnim pristupom, PostgreSQL uspešno podržava visoka radna opterećenja bez ugrožavanja integriteta podataka.

Ovaj rad ima za cilj da detaljno objasni koncepte transakcija i ACID izolacije u kontekstu PostgreSQL-a. Počevši od osnova — šta je transakcija i zašto je neophodna — progresivno se istražuju sve složeniji aspekti: ACID svojstva, fenomeni konkurentnog izvršavanja, nivoi izolacije, MVCC arhitektura, mehanizmi zaključavanja, kao i praktična demonstracija navedenih koncepata u realnom okruženju.

2. Transakcije u relacionim bazama podataka

2.1. Definicija i osnovna svojstva

Koncept transakcija u bazama podataka razvijen je tokom 1970-ih godina, paralelno sa razvojem prvih relacionih sistema. Jim Gray, dobitnik Turingove nagrade, definisao je i formalizovao osnovna svojstva transakcija i mehanizme oporavka koji su i danas u upotrebi. Termin ACID uveo je Andreas Reuter zajedno sa Theom Härderom 1983. godine u radu "Principles of Transaction-Oriented Database Recovery", koji je postavio temelje za sve moderne transakcione sisteme. PostgreSQL, čiji razvoj počinje 1986. godine na Univerzitetu Berkli pod nazivom POSTGRES, od samog početka je projektovan sa punom podrškom za transakcije, a danas se smatra jednom od referentnih implementacija ACID principa u svetu softvera otvorenog koda.

Transakcija u kontekstu baza podataka predstavlja logičku jedinicu rada koja se sastoji od jedne ili više operacija nad podacima. Ključna karakteristika transakcije je da se ona tretira kao nedeljiva celina — ili se sve operacije u okviru transakcije izvršavaju uspešno i njihovi rezultati trajno čuvaju, ili se nijedna od operacija ne primenjuje na podatke.

Ovaj koncept je od ključne važnosti u okruženjima gde više korisnika ili aplikacija istovremeno pristupa istim podacima. Bez mehanizma transakcija, sistem bi bio podložan raznim vrstama grešaka i nekonzistentnosti koje bi mogle imati ozbiljne posledice — od finansijskih gubitaka do gubitka kritičnih medicinskih podataka.

Transakcija prolazi kroz nekoliko stanja tokom svog životnog ciklusa. Počinje u aktivnom stanju, kada se izvršavaju operacije. Potom može biti parcijalno potvrđena (kada su sve operacije izvršene ali još nisu trajno sačuvane), potvrđena (COMMIT — sve promene su trajno sačuvane), ili poništena (ROLLBACK — sve promene su poništene i sistem se vraća u prethodno konzistentno stanje).

2.2. Životni ciklus transakcije

Svaka transakcija prolazi kroz jasno definisan životni ciklus koji se može opisati sledećim fazama:

- Početno stanje: Transakcija se inicira naredbom BEGIN ili START TRANSACTION. U ovom trenutku sistem beleži početak transakcije i dodeljuje joj jedinstveni identifikator.
- Aktivno stanje: Transakcija izvršava niz operacija — SELECT, INSERT, UPDATE, DELETE. Sve promene su privremene i vidljive su samo unutar te transakcije (zavisno od nivoa izolacije).
- Parcijalno potvrđeno stanje: Kada su sve planirane operacije izvršene, transakcija ulazi u ovo stanje neposredno pre finalnog COMMIT-a. Sistem proverava da li su sve operacije bile uspešne.
- Potvrđeno stanje (Committed): COMMIT naredba trajno čuva sve promene u bazi. Od ovog trenutka, promene su vidljive svim ostalim transakcijama.
- Poništeno stanje (Aborted): Ako dođe do greške ili se eksplicitno pozove ROLLBACK, sistem poništava sve promene i vraća se u stanje koje je postojalo pre početka transakcije.

Važno je napomenuti da PostgreSQL podržava i delimično poništavanje transakcija kroz mehanizam sačuvanih tačaka (SAVEPOINT). Ovo omogućava fino granularnu kontrolu nad tim koji delovi transakcije se poništavaju u slučaju greške.

2.3. Transakcije u PostgreSQL-u - osnovna sintaksa

PostgreSQL pruža bogatu sintaksu za rad sa transakcijama. Svaka transakcija započinje ključnom rečju BEGIN i završava se sa COMMIT ili ROLLBACK. Primer prenosa novca između dva računa:

```
BEGIN;  
UPDATE racuni SET stanje = stanje - 1000 WHERE id = 1;  
UPDATE racuni SET stanje = stanje + 1000 WHERE id = 2;  
COMMIT;
```

U slučaju greške, sve promene se poništavaju naredbom ROLLBACK. PostgreSQL takođe podržava SAVEPOINT mehanizam koji omogućava delimično poništavanje — korisno kada transakcija obuhvata više logičkih celina od kojih samo jedna ne uspe.

```
BEGIN;  
INSERT INTO narudzbe VALUES (1, 'Artikal A', 100);  
SAVEPOINT tacka1;  
INSERT INTO narudzbe VALUES (2, 'Artikal B', 200);  
ROLLBACK TO SAVEPOINT tacka1; - poništava samo drugi INSERT  
COMMIT; - čuva samo prvi INSERT
```

3. ACID svojstva

ACID je akronim koji opisuje četiri ključna svojstva koja transakcioni sistem baze podataka mora da garantuje. Ova svojstva su formulisana kasnih 1970-ih i ranih 1980-ih godina i danas predstavljaju standard koji svaki ozbiljan relacioni sistem mora da ispuni. Ime ACID dolazi od početnih slova

engleskih reči: Atomicity (Atomičnost), Consistency (Konzistentnost), Isolation (Izolacija) i Durability (Trajnost).

3.1. Atomičnost (Atomicity)

Atomičnost garantuje da se transakcija tretira kao nedeljiva (atomarna) jedinica izvršavanja. To znači da se ili sve operacije unutar transakcije uspešno izvrše, ili se nijedna ne primeni na trajno stanje baze podataka. Ne postoji delimično izvršena transakcija.

PostgreSQL implementira atomičnost kroz Write-Ahead Log (WAL) mehanizam. Pre nego što se bilo koja promena primeni na stvarne podatke, PostgreSQL beleži tu promenu u WAL dnevnik. Ako dođe do pada sistema, PostgreSQL može koristiti WAL dnevnik za oporavak — ili ponavljanjem operacija koje su bile deo potvrđenih transakcija, ili zanemarivanjem operacija koje nisu bile potvrđene.

Praktičan primer atomičnosti jeste bankarski prenos novca — smanjenje iznosa na jednom računu i povećanje na drugom moraju se desiti zajedno ili nijedno. Nema scenarija u kome je novac uzet sa jednog računa, ali nije dodat na drugi.

3.2. Konzistentnost (Consistency)

Konzistentnost garantuje da transakcija prevodi bazu podataka iz jednog konzistentnog stanja u drugo konzistentno stanje. Konzistentnost je u prvom redu odgovornost programera — oni moraju definisati pravila integriteta koja osiguravaju da podaci uvek budu u validnom stanju.

PostgreSQL podržava više mehanizama za osiguravanje konzistentnosti: PRIMARY KEY ograničenja koja osiguravaju jedinstvenost identifikatora, FOREIGN KEY ograničenja koja osiguravaju referencijalni integritet, CHECK ograničenja koja validiraju vrednosti podataka, NOT NULL ograničenja, kao i okidače i proceduralne funkcije za složena pravila. Na primer, zabrana negativnog stanja na računu može se definisati kao CHECK ograničenje:

```
ALTER TABLE racuni ADD CONSTRAINT provera_stanja CHECK (stanje >= 0);
```

3.3. Izolacija (Isolation)

Izolacija garantuje da se svaka transakcija izvršava kao da je jedina u sistemu, čak i kada se više transakcija izvršava istovremeno. Efekti jedne transakcije nisu vidljivi drugoj transakciji dok se ta transakcija ne potvrdi, u zavisnosti od nivoa izolacije.

Izolacija je možda najkompleksnije od četiri ACID svojstva, jer postoji fundamentalni kompromis između stepena izolacije i performansi sistema. Viši nivo izolacije daje veću ispravnost ali smanjuje propusnost sistema. PostgreSQL implementira različite nivoe izolacije koji omogućavaju programerima da biraju pravi balans za svoju aplikaciju. Detaljna analiza nivoa izolacije data je u poglavlju 5 ovog rada.

3.4. Trajnost (Durability)

Trajnost garantuje da jednom kada je transakcija potvrđena (COMMIT), njene promene ostaju trajne — čak i u slučaju pada sistema, nestanka struje ili drugog katastrofalnog događaja. Potvrđene promene moraju preživeti bilo koji tip sistemskog kvara.

PostgreSQL implementira trajnost primarno kroz Write-Ahead Log (WAL). Svaka promena se prvo

zapisuje u WAL datoteku pre nego što se primeni na podatkovne datoteke. WAL datoteke se čuvaju na disku, a PostgreSQL osigurava da su fizički sačuvane (fsync) pre nego što potvrdi transakciju klijentu. Na ovaj način, čak i u slučaju iznenadnog pada sistema, PostgreSQL može pri sledećem pokretanju rekonstruisati sve potvrđene promene čitanjem WAL dnevnika.

U distribuiranim sistemima, trajnost je još kompleksnija — promene moraju biti replicirane na dovoljan broj čvorova pre nego što se smatraju trajnim. PostgreSQL podržava sinhronu replikaciju koja garantuje da su promene zapisane na standby serveru pre potvrde transakcije klijentu. Ovo pruža dodatni nivo zaštite u slučaju pada primarnog čvora, jer standby server u svakom trenutku poseduje potpuno ažurnu kopiju podataka.

ACID svojstvo	Šta garantuje	PostgreSQL implementacija
Atomičnost	Sve ili ništa	Write-Ahead Log + rollback
Konzistentnost	Validno stanje pre i posle	Constraints, triggers
Izolacija	Transakcije se ne ometaju	MVCC + nivoi izolacije
Trajnost	Commit je permanentan	WAL + fsync + checkpoint

4. Fenomeni pri konkurentnom izvršavanju

Kada se više transakcija izvršava istovremeno, mogu se javiti različiti problemi koji ugrožavaju ispravnost podataka. SQL standard definiše četiri takva fenomena, a nivoi izolacije se upravo definišu na osnovu toga koji fenomeni su dozvoljeni, a koji sprečeni.

4.1. Prljavo čitanje (Dirty Read)

Prljavo čitanje nastaje kada transakcija T2 čita podatak koji je izmenjen od strane transakcije T1, ali T1 još uvek nije potvrđena. Ako T1 potom uradi ROLLBACK, T2 je radila sa podacima koji nikada nisu postojali u konzistentnom stanju baze.

Primer: transakcija T1 smanjuje stanje računa sa 1.000 na 500 dinara. Transakcija T2 čita stanje i vidi 500 dinara, pa odlučuje da odobri kredit. T1 radi ROLLBACK i stanje se vraća na 1.000 dinara. T2 je odluku donela na osnovu nepostojećeg stanja.

PostgreSQL u praksi ne dozvoljava prljavo čitanja čak ni na najnižem nivou izolacije. Read Uncommitted se ponaša identično kao Read Committed, što je svesna odluka dizajnera koji su procenili da prljavo čitanja nemaju opravdanje ni u jednom realnom scenariju.

4.2. Neponovljivo čitanje (Non-repeatable Read)

Neponovljivo čitanje nastaje kada transakcija T1 čita isti red dva puta i dobija različite vrednosti, jer je transakcija T2 izmenila i potvrdila taj red između dva čitanja T1.

Primer: T1 čita cenu artikla (100 din). T2 menja cenu na 150 din i potvrđuje. T1 ponovo čita cenu i dobija 150 din. Unutar iste transakcije T1 je dobila dve različite vrednosti za isti red. Ovaj fenomen je problematičan u scenarijima gde aplikacija čita iste podatke više puta i očekuje konzistentnost, na primer pri izračunavanju ukupnih iznosa.

4.3. Fantomsko čitanje (Phantom Read)

Fantomsko čitanje nastaje kada transakcija T1 izvršava isti upit dva puta i dobija različit skup redova, jer je transakcija T2 ubacila ili obrisala redove koji odgovaraju uslovu upita T1.

Primer: T1 broji broj zaposlenih u odeljenju i dobija 10. T2 zapošljava novog radnika u isto odeljenje i potvrđuje. T1 ponovo broji i dobija 11 — pojavio se fantomski red. Razlika između neponovljivog i fantomskog čitanja je u tome što se neponovljivo čitanje odnosi na izmenu postojećih redova, dok se fantomsko čitanje odnosi na pojavu novih ili nestanak postojećih redova.

4.4. Serializaciona anomalija

Serijalizaciona anomalija je situacija u kojoj kombinovani rezultat paralelno izvršenih transakcija nije ekvivalentan nijednom redosledu u kome bi te iste transakcije bile izvršene jedna po jedna (serijalno). Ovo je najsloženiji fenomen i jedini koji nije moguće sprečiti na nivou Repeatable Read — potreban je Serializable nivo. PostgreSQL implementira Serializable nivo kroz SSI (Serializable Snapshot Isolation) algoritam koji detektuje i prevenira serijalizacione anomalije uz minimalan uticaj na performanse.

Fenomen	Uzrok	Nivo koji sprečava
Prljavo čitanje	Čitanje uncommitted podataka	Read Committed i viši
Neponovljivo čitanje	Promena reda između dva SELECT-a	Repeatable Read i viši
Fantomsko čitanje	Dodavanje/brisanje redova između upita	Serializable (i Rep. Read u PG)
Serijalizaciona anomalija	Neekvivalentan redosled transakcija	Serializable

5. Nivoi izolacije u PostgreSQL-u

SQL standard definiše četiri nivoa izolacije transakcija. Svaki nivo definiše koji fenomeni su dozvoljeni, a koji sprečeni. Viši nivo izolacije pruža veću sigurnost ali obično dolazi po cenu performansi. PostgreSQL podržava sve četiri nivoa, ali sa specifičnim implementacionim detaljima koji ga razlikuju od striktno definicije SQL standarda.

Nivo izolacije	Prljavo čitanje	Neponovljivo čitanje	Fantomsko čitanje	Serijalizaciona anomalija
Read Uncommitted	Sprečeno*	Moguće	Moguće	Moguće
Read Committed	Sprečeno	Moguće	Moguće	Moguće
Repeatable Read	Sprečeno	Sprečeno	Sprečeno*	Moguće
Serializable	Sprečeno	Sprečeno	Sprečeno	Sprečeno

* PostgreSQL tretira Read Uncommitted identično kao Read Committed. Repeatable Read u PostgreSQL-u sprečava i fantomsko čitanje, što je strože od SQL standarda.

5.1. Read Uncommitted

Read Uncommitted je najniži nivo izolacije prema SQL standardu i teorijski dozvoljava prljavo čitanje. Međutim, PostgreSQL ne implementira stvarno Read Uncommitted ponašanje — ovaj nivo se ponaša identično kao Read Committed. Razlog je jednostavan: prljavo čitanje su retko korisna u praksi i gotovo uvek dovode do grešaka u aplikacijama.

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

5.2. Read Committed

Read Committed je podrazumevani nivo izolacije u PostgreSQL-u i najčešći izbor za većinu aplikacija. Na ovom nivou, svaki upit unutar transakcije vidi samo podatke koji su bili potvrđeni u trenutku početka tog upita, ne početka transakcije. To znači da dva upita unutar iste transakcije mogu videti različite podatke ako je neka druga transakcija izmenila i potvrdila podatke između ta dva upita — ovo omogućava neponovljiva čitanja i fantomska čitanja.

Read Committed pruža dobar balans između izolacije i performansi i prikladan je za većinu OLTP radnih opterećenja gde su transakcije kratke i jednostavne.

```
BEGIN;
```

```
SELECT stanje FROM racuni WHERE id = 1;
```

Druga transakcija može izmeniti red između ova dva upita;

```
SELECT stanje FROM racuni WHERE id = 1; - može videti drugačije vrednosti  
COMMIT;
```

U praksi, Read Committed je prikladan za većinu web aplikacija i OLTP sistema — online prodavnice, sisteme za rezervacije, društvene mreže. U ovim sistemima transakcije su kratke, operacije jednostavne, a blago nekonzistentno čitanje retko ima praktične posledice. Većina programera koji rade sa PostgreSQL-om nikada ne mora da menja podrazumevani nivo izolacije.

5.3. Repeatable Read

Na nivou Repeatable Read, transakcija vidi konzistentnu snimku podataka iz trenutka kada je transakcija počela. Svi upiti unutar transakcije videće iste podatke, bez obzira na promene koje su u međuvremenu napravile i potvrdile druge transakcije. PostgreSQL implementacija Repeatable Read je stroža od SQL standarda — ona takođe sprečava fantomska čitanja, zahvaljujući MVCC arhitekturi koja prirodno pruža konzistentne snimke.

Repeatable Read je prikladan za dugotrajne transakcije koje moraju čitati više redova i zahtevaju konzistentnu sliku tokom čitavog procesa, kao što je generisanje finansijskih izveštaja.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

```
SELECT SUM(iznos) FROM transakcije WHERE datum = CURRENT_DATE;
```

- Čak i ako neko ubaci novi red, ovaj upit će videti isti rezultat

```
SELECT COUNT(*) FROM transakcije WHERE datum = CURRENT_DATE;  
COMMIT;
```

Tipični slučajevi upotrebe Repeatable Read nivoa su generisanje finansijskih izveštaja, obračun plata, izrada periodičnih analiza i sve situacije gde jedan proces mora da pročita više međusobno povezanih podataka i garantuje da su svi iz istog trenutka. Na primer, ako izveštaj sabira prihode iz više tabela, nije prihvatljivo da se između dva upita promeni kurs valute ili doda nova stavka.

5.4. Serializable

Serializable je najviši nivo izolacije i jedini koji garantuje potpunu zaštitu od svih anomalija. PostgreSQL garantuje da je rezultat konkurentnog izvršavanja transakcija identičan nekom mogućem

serijskom redosledu. Implementiran je kroz mehanizam Serializable Snapshot Isolation (SSI) koji detektuje konflikte zasnovane na zavisnostima između transakcija, bez blokiranja čitanja.

Prednost SSI-ja jeste da čitanje ne blokira pisanje ni na ovom nivou. Ipak, postoji overhead detekcije konflikata i mogućnost da transakcije budu abortirane, pa aplikacija mora biti spremna da ponovi transakciju kada dobije grešku o konfliktu serijalizacije.

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
```

- PostgreSQL detektuje serijalizacione konflikte i može abortirati transakciju:

```
ERROR: could not serialize access due to read/write dependencies
```

```
COMMIT;
```

Serializable nivo se koristi u sistemima gde je ispravnost kritičnija od performansi — bankarski sistemi, medicinska dokumentacija, sistemi za upravljanje zalihama gde negativna količina nije dozvoljena ni u kakvim okolnostima. Važno je napomenuti da aplikacije koje koriste ovaj nivo moraju implementirati logiku ponovnog pokušaja (retry) kada dobiju grešku o serijalizacionom konfliktu, što je standardna i očekivana praksa.

6. MVCC - Kontrola konkurentnog pristupa zasnovana na verzijama

Multi-Version Concurrency Control (MVCC) je tehnika upravljanja konkurentnim pristupom koja omogućava čitaocima da ne blokiraju pisce i obrnuto. Umesto zaključavanja redova pri čitanju, MVCC čuva više verzija istog reda, a svaka transakcija vidi odgovarajuću verziju na osnovu svog "snimka" stanja baze podataka u određenom trenutku.

Osnovna ideja je elegantno jednostavna: kada se red izmeni, PostgreSQL ne briše staru vrednost, već kreira novu verziju reda sa izmenjenim podacima. Stara verzija ostaje sačuvana sve dok postoji bar jedna aktivna transakcija koja je može videti. Na taj način, transakcija koja čita podatke uvek dobija konzistentnu sliku baze — onu kakva je bila u trenutku njenog početka — bez potrebe da čeka na završetak transakcija koje pišu.

6.1. Principi rada MVCC-a

Multi-Version Concurrency Control (MVCC) je tehnika upravljanja konkurentnim pristupom podacima koja umesto zaključavanja redova čuva više verzija istog reda istovremeno. Osnovna ideja je da svaka transakcija dobija konzistentnu sliku baze podataka kakva je bila u određenom trenutku, bez blokiranja ostalih transakcija.

Ključna prednost MVCC-a nad pristupom zasnovanim na zaključavanju jeste da operacije čitanja nikada ne blokiraju operacije pisanja, i obrnuto. Dok jedna transakcija ažurira red, druga transakcija može istovremeno čitati prethodnu verziju tog istog reda, bez čekanja. Ovo dramatično povećava propusnost sistema u okruženjima sa mešovitim workload-om.

Da bismo ilustrovali kako MVCC funkcioniše u praksi, zamislimo sledeći scenario: transakcija T1 čita red sa stanjem računa (vrednost: 1.000 dinara) u trenutku t1. Istovremeno, transakcija T2 menja to stanje na 2.000 dinara i potvrđuje promenu u trenutku t2. Bez MVCC-a, T1 bi morala da čeka dok T2 ne završi, ili bi videla nekonzistentne podatke. Sa MVCC-om, T1 i dalje vidi vrednost 1.000 dinara — svoju verziju reda — dok T2 kreira novu verziju sa vrednošću 2.000 dinara. Obe verzije koegzistiraju u bazi sve dok T1 ne završi, nakon čega stara verzija postaje kandidat za uklanjanje kroz VACUUM

proces.

6.2. Vidljivost verzija u PostgreSQL-u

PostgreSQL implementira MVCC dodavanjem skrivenih sistemskih kolona svakom redu u bazi podataka. Dve najvažnije su xmin i xmax. Kolona xmin čuva ID transakcije koja je kreirala tu verziju reda (INSERT ili UPDATE), dok xmax čuva ID transakcije koja je obrisala ili ažurirala taj red — vrednost 0 znači da je red još uvek aktivan.

Kada transakcija čita podatke, PostgreSQL proverava xmin i xmax vrednosti svakog reda i određuje da li je ta verzija vidljiva za tekuću transakciju. Red je vidljiv ako je transakcija koja ga je kreirala bila potvrđena pre nego što je tekuća transakcija počela, i ako transakcija koja ga je obrisala ili nije bila potvrđena ili je potvrđena posle početka tekuće transakcije.

Kada se red ažurira, PostgreSQL ne menja originalni red, već kreira novu verziju sa novim vrednostima. Stara verzija ostaje u bazi dok ima transakcija koje je mogu videti.

- Prikaz skrivenih MVCC kolona
SELECT xmin, xmax, ctid, * FROM racuni;

Konkretna primer vidljivosti verzija možemo demonstrirati direktnim upitom nad sistemskim kolonama:

```
SELECT xmin, xmax, id, vlasnik, stanje FROM racuni;
```

Rezultat pokazuje xmin vrednost za svaki red — identifikator transakcije koja je kreirala tu verziju. Nakon izvršavanja UPDATE naredbe, PostgreSQL kreira novi red sa novim xmin vrednostima, dok stari red dobija xmax vrednost koja označava transakciju koja ga je "obrisala". Red sa xmax = 0 je aktivan i nema naslednika. Ovo je suštinski mehanizam koji omogućava konzistentne snimke bez blokiranja.

Vidljivost konkretnog reda za transakciju T određena je sledećim pravilima: xmin transakcija mora biti potvrđena pre nego što je T počela, i xmax transakcija ili ne postoji, ili nije bila potvrđena pre početka T. Sve ostale verzije su za T nevidljive, bez obzira na to da li postoje u bazi.

6.3. Vacuum i čišćenje starih verzija

Nagomilavanje starih verzija redova koje više nijedna transakcija ne vidi naziva se table bloat i može negativno uticati na performanse. PostgreSQL rešava ovaj problem kroz VACUUM proces koji uklanja zastarele verzije redova i oslobađa prostor.

Postoje dve vrste VACUUM operacije. Obična VACUUM operacija uklanja zastarele verzije redova i označava taj prostor kao slobodan za ponovnu upotrebu, ali ne vraća prostor operativnom sistemu. VACUUM FULL kreira kompaktnu verziju tabele i vraća oslobođeni prostor operativnom sistemu, ali zahteva ekskluzivni zaključak na tabeli tokom izvršavanja. PostgreSQL uključuje autovacuum proces koji automatski pokreće VACUUM operacije u pozadini na osnovu konfigurabilnih pragova.

- Ručno pokretanje VACUUM-a
VACUUM ANALYZE racuni;
- Statistike o mrtvim verzijama
SELECT relname, n_dead_tup, n_live_tup, last_autovacuum
FROM pg_stat_user_tables WHERE relname = 'racuni';

Bez redovnog vakuumiranja, tabele bi se neprestano širile zbog nagomilavanja starih verzija redova — ovaj problem se naziva table bloat. U ekstremnim slučajevima, table bloat može toliko usporiti sistem da indeksi postaju neupotrebljivi, a skeniranje tabele traje neprihvatljivo dugo.

PostgreSQL autovacuum daemon automatski prati broj mrtvih verzija po tabeli i pokreće VACUUM kada pređu konfigurisani prag. Prag se može podesiti po tabeli:

```
ALTER TABLE racuni SET (autovacuum_vacuum_scale_factor = 0.01);
```

Ovo znači da će autovacuum biti pokrenut čim 1% redova tabele postane mrtav, umesto podrazumevanog 20%. Za tabele sa intenzivnim pisanjem ovo je preporučena praksa.

7. Zaključavanje i deadlock

Iako MVCC smanjuje potrebu za zaključavanjem pri citanju, PostgreSQL jos uvek koristi zaključke u različite svrhe. Razumevanje mehanizma zaključavanja je ključno za pisanje efikasnih i korektnih aplikacija.

7.1. Vrste zaključavanja

PostgreSQL koristi hijerarhijski sistem zaključavanja sa osam različitih režima na nivou tabele. Ključno svojstvo ovog sistema je kompatibilnost — dva zaključka su kompatibilna ako mogu istovremeno biti aktivni na istom objektu, a nekompatibilni ako jedan mora čekati na drugi.

Sledeća tabela prikazuje kompatibilnost najvažnijih režima zaključavanja:

Režim	ACCESS SHARE	ROW SHARE	ROW EXCLUSIVE	SHARE	EXCLUSIVE	ACCESS EXCLUSIVE
ACCESS SHARE	✓	✓	✓	✓	✓	✗
ROW SHARE	✓	✓	✓	✓	✗	✗
ROW EXCLUSIVE	✓	✓	✓	✗	✗	✗
SHARE	✓	✓	✗	✓	✗	✗
EXCLUSIVE	✓	✗	✗	✗	✗	✗
ACCESS EXCLUSIVE	✗	✗	✗	✗	✗	✗

✓ — kompatibilni ✗ — konflikt (zahtev čeka)

Najvažnije zaključke u svakodnevnom radu koriste sledeće operacije: SELECT automatski postavlja ACCESS SHARE zaključak (najblažu vrstu), INSERT/UPDATE/DELETE postavljaju ROW EXCLUSIVE zaključak, dok DDL operacije poput ALTER TABLE i DROP TABLE postavljaju ACCESS EXCLUSIVE zaključak koji je nekompatibilan sa svim ostalim — zbog toga ALTER TABLE čeka da se završe sve aktivne transakcije pre nego što može da se izvrši.

PostgreSQL koristi različite vrste zaključavanja u zavisnosti od operacije i granularnosti. Zaključci na nivou tabele koriste se za operacije koje utiču na čitavu tabelu — ALTER TABLE, TRUNCATE, DROP TABLE — i PostgreSQL ima osam različitih režima zaključavanja tabele sa različitim stepenima kompatibilnosti. Zaključci na nivou reda automatski se postavljaju pri operacijama pisanja, dok čitanje ne postavlja zaključke zahvaljujući MVCC mehanizmu.

PostgreSQL podržava i eksplicitne zaključke na nivou reda kroz SELECT FOR UPDATE i SELECT FOR SHARE, koji omogućavaju programerima da eksplicitno zaključaju redove pri čitanju kada žele da spreče konkurentne izmene. Pored toga, savetodavni zaključci (Advisory locks) su aplikaciono-specifični zaključci koje programeri postavljaju i oslobađaju ručno, korisni za implementaciju distribuiranih kritičnih sekcija.

- Eksplicitno zaključavanje reda:
SELECT * FROM proizvodi WHERE id = 1 FOR UPDATE;
- Bez čekanja — vraća grešku ako ne može zaključati:
SELECT * FROM proizvodi WHERE id = 1 FOR UPDATE NOWAIT;
- Pregled aktivnih zaključaka:
SELECT pid, locktype, relation::regclass, mode, granted
FROM pg_locks WHERE NOT granted;

Posebno važna napomena za praksu: SELECT FOR UPDATE postavlja ROW SHARE zaključak na nivou tabele i ekskluzivni zaključak na nivou reda. To znači da dva paralelna SELECT FOR UPDATE na različitim redovima iste tabele neće blokirati jedan drugog — zaključci na nivou reda su nezavisni, pod uslovom da se radi o različitim redovima.

7.2. Mrtva petlja (Deadlock)

Deadlock nastaje kada dve ili više transakcija čekaju jedna na drugu da oslobode zaključke, stvarajući cirkularne zavisnosti iz kojih nema izlaza bez spoljne intervencije. Klasičan primer: transakcija T1 zaključava red A i čeka na red B, dok transakcija T2 zaključava red B i čeka na red A. Nijedna od transakcija ne može napredovati.

- T1 (Sesija 1):
BEGIN;
UPDATE racuni SET stanje = stanje - 100 WHERE id = 1; - zaključava red 1
UPDATE racuni SET stanje = stanje + 100 WHERE id = 2; - čeka na red 2
- T2 (Sesija 2, paralelno):
BEGIN;
UPDATE racuni SET stanje = stanje - 100 WHERE id = 2; - zaključava red 2
UPDATE racuni SET stanje = stanje + 100 WHERE id = 1; - čeka na red 1 → DEADLOCK

PostgreSQL automatski detektuje deadlock-ove periodičnom proverom grafa čekanja. Kada detektuje deadlock, poništava jednu od transakcija i vraća grešku: ERROR: deadlock detected.

7.3. Prevencija i rešavanje deadlock-a

Najdelotvornija strategija je prevencija deadlock-a kroz disciplinovano kodiranje. Konzistentan redosled zaključavanja — uvek pristupati resursima u istom redosledu u svim transakcijama — eliminiše mogućnost kružnih zavisnosti. Kratke transakcije smanjuju verovatnoću konflikta jer se zaključci drže kraće. Zaključavanje svih potrebnih redova na početku transakcije kroz SELECT FOR UPDATE, umesto postepenog zaključavanja, takođe smanjuje rizik.

Aplikacija uvek treba da bude pripremljena da ponovi transakciju kada dobije grešku o deadlock-u. Ovo je posebno važno na Serializable nivou izolacije gde PostgreSQL može abortirati transakciju i zbog serijalizacionih konflikata.

Pored konzistentnog redosleda zaključavanja, postoje i druge tehnike prevencije deadlocka. Podešavanje lock_timeout parametra osigurava da transakcija ne čeka beskonačno:

sql

```
SET lock_timeout = '5s';
```

Praćenje potencijalnih deadlockova moguće je kroz sistemski pogled pg_stat_activity kombinovan sa pg_blocking_pids funkcijom, što smo demonstrirali u praktičnom delu. Za produkcijska okruženja,

preporučuje se i podešavanje `deadlock_timeout` parametra u `postgresql.conf` — podrazumevana vrednost je 1 sekunda, što znači da PostgreSQL proverava da li postoji deadlock tek nakon što transakcija čeka više od jedne sekunde.

8. Praktični primeri

Pre demonstracije praktičnih primera, kreirana je testna baza podataka koja simulira jednostavan bankarski sistem. Definisane su četiri tabele: `racuni` (bankovni računi korisnika), `artikli` (proizvodi u sistemu), `narudzbe` (veze između narudžbina i artikala) i `log_transakcija` (evidencija događaja).

Tabela `racuni` sadrži `CHECK` ograničenje koje sprečava negativno stanje računa, dok tabela `narudzbe` sadrži `FOREIGN KEY` koji osigurava referencijalni integritet prema tabeli `artikli`. Ova ograničenja su deo mehanizma konzistentnosti koji će biti demonstriran u narednim testovima.

Početni podaci prikazuju tri korisnika sa različitim stanjima računa (Tabela 1), kao i tri artikla sa odgovarajućim količinama i cenama (Tabela 2)

```
DROP TABLE IF EXISTS racuni CASCADE;
DROP TABLE IF EXISTS narudzbe CASCADE;
DROP TABLE IF EXISTS artikli CASCADE;
DROP TABLE IF EXISTS log_transakcija CASCADE;
```

```
CREATE TABLE racuni (
    id      SERIAL PRIMARY KEY,
    vlasnik VARCHAR(100) NOT NULL,
    stanje  NUMERIC(12,2) NOT NULL,
    CONSTRAINT pozitivno_stanje CHECK (stanje >= 0)
);
```

```
CREATE TABLE artikli (
    id      SERIAL PRIMARY KEY,
    naziv   VARCHAR(100),
    kolicina INTEGER NOT NULL,
    cena    NUMERIC(10,2),
    CONSTRAINT pozitivna_kolicina CHECK (kolicina >= 0)
);
```

```
CREATE TABLE narudzbe (
    id      SERIAL PRIMARY KEY,
    artikal_id INTEGER REFERENCES artikli(id),
    kolicina INTEGER,
    vreme   TIMESTAMP DEFAULT NOW()
```

);

```
CREATE TABLE log_transakcija (  
    id SERIAL PRIMARY KEY,  
    poruka TEXT,  
    vreme TIMESTAMP DEFAULT NOW()  
);
```

```
INSERT INTO racuni (vlasnik, stanje) VALUES  
('Ana', 10000.00),  
('Bojan', 5000.00),  
('Carla', 2000.00);
```

```
INSERT INTO artikli (naziv, kolicina, cena) VALUES  
('Laptop', 10, 1200.00),  
('Mis', 100, 25.00),  
('Monitor', 15, 350.00);
```

```
SELECT 'SETUP USPESNO!' AS status;  
SELECT * FROM racuni;  
SELECT * FROM artikli;
```

Data Output Messages Notifications

	id [PK] integer	naziv character varying (100)	kolicina integer	cena numeric (10,2)
1	1	Laptop	10	1200.00
2	2	Mis	100	25.00
3	3	Monitor	15	350.00
	id [PK] integer	vlasnik character varying (100)	stanje numeric (12,2)	
1	1	Ana	10000.00	
2	2	Bojan	5000.00	
3	3	Carla	2000.00	

8.1 Commit i Rollback:

U prvom praktičnom testu demonstrirano je osnovno ponašanje transakcija kroz dva scenarija: uspešan prenos sredstava (COMMIT) i poništeni prenos (ROLLBACK).

Query

Query History

AI Assistant

1

BEGIN;

2

UPDATE racuni **SET** stanje = stanje - 1000 **WHERE** id = 1;

3

UPDATE racuni **SET** stanje = stanje + 1000 **WHERE** id = 2;

4

SELECT id, vlasnik, stanje **FROM** racuni;

5

COMMIT;

6

7

SELECT id, vlasnik, stanje **FROM** racuni;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing rows: 1 to 3

✓

Page No: 1

	id [PK] integer	vlasnik character varying (100)	stanje numeric (12,2)
1	3	Carla	2000.00
2	1	Ana	9000.00
3	2	Bojan	6000.00

Query
Query History
AI Assistant

```

1 BEGIN;
2     UPDATE racuni SET stanje = stanje - 5000 WHERE id = 1;
3     UPDATE racuni SET stanje = stanje + 5000 WHERE id = 2;
4     SELECT id, vlasnik, stanje FROM racuni;
5 ROLLBACK;
6
7 SELECT id, vlasnik, stanje FROM racuni;

```

Data Output
Messages
Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 3

✓ ✕ 📄

Page No: 1

	id [PK] integer	vlasnik character varying (100)	stanje numeric (12,2)
1	3	Carla	2000.00
2	1	Ana	9000.00
3	2	Bojan	6000.00

U prvom scenariju, transakcija je smanjila stanje Aninog računa za 1.000 dinara i povećala stanje Bojanovog računa za isti iznos. Nakon izvršavanja naredbe COMMIT, promene su postale trajne — SELECT upit nakon transakcije potvrdio je nova stanja (Ana: 9.000, Bojan: 6.000).

U drugom scenariju, pokušao je prenos od 5.000 dinara. Unutar transakcije promene su bile vidljive, međutim izvršavanjem naredbe ROLLBACK sve izmene su poništene. Kao što se vidi na priloženom prikazu rezultata, stanja računa ostala su nepromenjena (Ana: 9.000, Bojan: 6.000), što potvrđuje da PostgreSQL garantuje atomičnost — transakcija koja nije potvrđena ne ostavlja nikakav trag u bazi podataka.

8.2 Delimično poništavanje pomoću SAVEPOINT mehanizma

[Query](#) [Query History](#) [AI Assistant](#)

```
1 BEGIN;
2     INSERT INTO log_transakcija (poruka) VALUES ('Korak 1 - zapoceto');
3     SAVEPOINT tacka1;
4
5     INSERT INTO log_transakcija (poruka) VALUES ('Korak 2 - sredina');
6     SAVEPOINT tacka2;
7
8     INSERT INTO log_transakcija (poruka) VALUES ('Korak 3 - GRESKA!');
9     ROLLBACK TO SAVEPOINT tacka2;
10
11     INSERT INTO log_transakcija (poruka) VALUES ('Korak 3 - ispravka');
12 COMMIT;
13
14 SELECT * FROM log_transakcija;
```

Data Output Messages Notifications

+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows:

1

to

3

✓

✎

📄

Page No:

1

of 1

⏪

⏩

	id [PK] integer ✎	poruka text ✎	vreme timestamp without time zone ✎
1	1	Korak 1 - zapoceto	2026-06-06 10:43:56.325681
2	2	Korak 2 - sredina	2026-06-06 10:43:56.325681
3	4	Korak 3 - ispravka	2026-06-06 10:43:56.325681

PostgreSQL omogućava fino granularnu kontrolu nad transakcijama kroz mehanizam čuvanih tačaka (SAVEPOINT). Za razliku od potpunog ROLLBACK-a koji poništava celu transakciju, ROLLBACK TO SAVEPOINT poništava samo operacije izvršene nakon određene tačke, dok sve prethodne ostaju na snazi.

U demonstriranom primeru, transakcija je upisivala poruke u tabelu log_transakcija u tri koraka. Nakon drugog koraka postavljena je čuvana tačka tacka2. Treći korak je simulirao grešku, nakon čega je izvršen povratak na tacka2. Umesto greške, upisana je ispravna poruka, a transakcija je potvrđena.

Rezultat prikazan na slici pokazuje tri upisane poruke sa ID-evima 1, 2 i 4 — ID 3 nedostaje jer je bio dodeljen poništenom unosu "Korak 3 - GRESKA!". Ovo ilustruje važnu karakteristiku PostgreSQL-a: SERIAL brojač se ne vraća unazad čak ni kada je transakcija poništena, što je očekivano i ispravno ponašanje sistema.

8.3 Konsistentnost (CHECK constraint):

Query	Query History	AI Assistant
1	BEGIN;	
2	UPDATE racuni SET stanje = stanje - 99999 WHERE id = 3;	
3	ROLLBACK;	
4		
5	SELECT id, vlasnik, stanje FROM racuni WHERE id = 3;	

Data Output	Messages	Notifications
	ERROR: new row for relation "racuni" violates check constraint "pozitivno_stanje" Failing row contains (3, Carla, -97999.00). SQL state: 23514 Detail: Failing row contains (3, Carla, -97999.00).	

Drugo svojstvo ACID modela, konzistentnost, garantuje da transakcija ne može dovesti bazu podataka u nevalidno stanje. U PostgreSQL-u se ovo postiže kroz ograničenja integriteta koja se definišu pri kreiranju tabele.

U demonstriranom primeru, pokušano je skidanje iznosa od 99.999 dinara sa Carlinog računa koji ima stanje od svega 2.000 dinara. Tabela racuni sadrži CHECK ograničenje pozitivno_stanje koje zabranjuje negativno stanje računa.

Kao što se vidi na prikazanoj poruci greške, PostgreSQL je odmah odbio operaciju sa porukom: "new row for relation racuni violates check constraint pozitivno_stanje", pri čemu je u detalju prikazao vrednost koja bi bila upisana (-97.999,00). Transakcija je automatski poništena i stanje Carlinog računa ostalo je nepromenjeno na 2.000 dinara.

Ovo potvrđuje da konzistentnost nije samo odgovornost aplikacije — baza podataka sama brani pravila integriteta, bez obzira na to odakle dolazi zahtev za izmenom.

8.4 Foreign Key:

Pored CHECK ograničenja, konzistentnost se osigurava i kroz FOREIGN KEY ograničenja koja čuvaju referencijalni integritet između tabela. Ovo sprečava situaciju u kojoj bi zapis u jednoj tabeli referencirao nepostojeći zapis u drugoj.

Query	Query History	AI Assistant
1	BEGIN;	
2	INSERT INTO narudzbe (artikal_id, kolicina) VALUES (999, 5);	
3	ROLLBACK;	

Data Output	Messages	Notifications
ERROR: insert or update on table "narudzbe" violates foreign key constraint "narudzbe_artikal_id_fkey" Key (artikal_id)=(999) is not present in table "artikli".		
SQL state: 23503 Detail: Key (artikal_id)=(999) is not present in table "artikli".		

U demonstriranom primeru, pokušano je kreiranje narudžbine za artikal sa ID-em 999, koji ne postoji u tabeli artikli. PostgreSQL je odmah odbio operaciju sa porukom: *"insert or update on table narudzbe violates foreign key constraint narudzbe_artikal_id_fkey"*, uz detalj: *"Key (artikal_id)=(999) is not present in table artikli"*.

Oba primera — CHECK i FOREIGN KEY — pokazuju da PostgreSQL aktivno čuva konzistentnost podataka na nivou baze, nezavisno od aplikacionog koda. Ovo je posebno važno u sistemima gde više različitih aplikacija ili korisnika pristupa istoj bazi podataka.

8.5 Dirty Read (Izolacija):

Tab1:

Query	Query History	AI Assistant
1	BEGIN;	
2	UPDATE racuni SET stanje = 99999 WHERE id = 1;	

Data Output	Messages	Notifications
UPDATE 1		
Query returned successfully in 104 msec.		

Tab2:

Query

Query History

AI Assistant

1

SELECT id, vlasnik, stanje FROM racuni WHERE id = 1;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	id [PK] integer	vlasnik character varying (100)	stanje numeric (12,2)
1	1	Ana	9000.00

Tab1:

Query	Query History	AI Assistant
1	COMMIT;	
Data Output	Messages	Notifications
COMMIT		
Query returned successfully in 85 msec.		

Tab2:

Query

Query History

AI Assistant

1

SELECT id, vlasnik, stanje FROM racuni WHERE id = 1;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	id [PK] integer	vlasnik character varying (100)	stanje numeric (12,2)
1	1	Ana	99999.00

Treće ACID svojstvo, izolacija, garantuje da transakcija ne može videti nepotvrđene promene druge transakcije. Ovaj fenomen, poznat kao "prljavo čitanje" (dirty read), demonstriran je kroz dve paralelne sesije u pgAdmin-u.

U prvoj sesiji (Tab 1) pokrenuta je transakcija koja je promenila stanje Aninog računa sa 9.000 na 99.999 dinara, ali bez izvršavanja COMMIT naredbe. U drugom koraku, druga sesija (Tab 2) je odmah pročitala stanje istog računa. Kao što se vidi na priloženom prikazu, Tab 2 je prikazao vrednost 9.000 — staro, potvrđeno stanje — iako je Tab 1 imao aktivnu nepotvrđenu promenu.

Nakon što je Tab 1 izvršio COMMIT, Tab 2 je ponovo pročitao isti red i sada prikazuje vrednost 99.999. Promena je postala vidljiva tek u trenutku potvrde transakcije.

Ovaj test potvrđuje da PostgreSQL na podrazumevanom nivou izolacije (Read Committed) nikada ne dozvoljava čitanje nepotvrđenih podataka, čime se sprečava jedna od najopasnijih anomalija konkurentnog izvršavanja.

8.6 Non-Repeatable Read:

Tab1:

▼

▼

▼

▼

No limit

▼

▼

▼

E

Query

Query History

AI Assistant

1

BEGIN;

2

SELECT id, vlasnik, stanje FROM racuni WHERE id = 2;

Data Output

Messages

Notifications

≡+

▼

▼

↓

SQL

	id [PK] integer	vlasnik character varying (100)	stanje numeric (12,2)
1	2	Bojan	6000.00

Tab2:

Query

Query History

AI Assistant

1

UPDATE racuni SET stanje = 7000 WHERE id = 2;

2

COMMIT;

Data Output

Messages

Notifications

WARNING: there is no transaction in progress

COMMIT

Query returned successfully in 147 msec.

Tab1:

Query
Query History
AI Assistant

1
SELECT id, vlasnik, stanje FROM racuni WHERE id = 2;

Data Output
Messages
Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 1

✎

📧

Page No: 1 of 1

⏪

⏴

⏵

⏩

	id [PK] integer ✎	vlasnik character varying (100) ✎	stanje numeric (12,2) ✎
1	2	Bojan	7000.00

Na podrazumevanom nivou izolacije Read Committed, moguća je pojava fenomena neponovljivog čitanja — situacije u kojoj isti upit unutar jedne transakcije vraća različite rezultate pri uzastopnim izvršavanjima.

U demonstriranom testu, Tab 1 je pokrenuo transakciju i pročitao stanje Bojanovog računa — rezultat je bio 6.000 dinara. Dok je ta transakcija još bila otvorena, Tab 2 je izmenio stanje na 7.000 dinara i potvrdio promenu. Kada je Tab 1 ponovo pročitao isti red, dobio je vrednost 7.000 dinara — različitu od one koju je video na početku iste transakcije.

Ovo je karakteristično ponašanje nivoa Read Committed: svaki upit unutar transakcije vidi najnovije potvrđene podatke u trenutku izvršavanja tog upita, a ne u trenutku početka transakcije. U scenarijima gde je konzistentnost podataka tokom cele transakcije kritična — na primer pri generisanju finansijskih izveštaja — ovaj fenomen može dovesti do netačnih rezultata.

8.7 Repeatable Read:

Prvo resetujemo:

```
UPDATE racuni SET stanje = 5000 WHERE id = 2;
```

Tab1:

Query

Query History

AI Assistant

1

2

BEGIN ISOLATION LEVEL REPEATABLE READ;

SELECT id, vlasnik, stanje FROM racuni WHERE id = 2;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

id

[PK] integer

vlasnik

character varying (100)

stanje

numeric (12,2)

1

2

Bojan

5000.00

Tab2:

Query Query History AI Assistant

1 UPDATE racuni SET stanje = 8000 WHERE id = 2;

2 COMMIT;

Data Output Messages Notifications

WARNING: there is no transaction in progress

COMMIT

Query returned successfully in 100 msec.

Tab1:

Query

Query History

AI Assistant

1

SELECT id, vlasnik, stanje FROM racuni WHERE id = 2;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	id [PK] integer	vlasnik character varying (100)	stanje numeric (12,2)
1	2	Bojan	5000.00

Za razliku od Read Committed nivoa, Repeatable Read garantuje da će svaki upit unutar transakcije videti iste podatke tokom čitavog njenog trajanja — bez obzira na promene koje su u međuvremenu napravile i potvrdile druge transakcije.

U demonstriranom testu, Tab 1 je pokrenuo transakciju sa Repeatable Read nivoom izolacije i pročitao stanje Bojanovog računa — rezultat je bio 5.000 dinara. Zatim je Tab 2 promenio stanje na 8.000 dinara i potvrdio promenu. Kada je Tab 1 ponovo izvršio isti SELECT upit unutar iste transakcije, rezultat je bio i dalje 5.000 dinara.

PostgreSQL je Tab 1 transakciji prikazao konzistentnu "snimku" podataka iz trenutka kada je transakcija počela, potpuno ignorišući promene koje su se desile u međuvremenu. Ovo je ključna razlika u odnosu na Read Committed nivo demonstriran u prethodnom testu, gde je isti scenario rezultovao različitim vrednostima.

8.8 Phantom Read:

Tab1:

Query
Query History
AI Assistant

1
2

BEGIN;
SELECT COUNT(*) AS broj_artikala FROM artikli;

Data Output
Messages
Notifications

≡+

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

broj_artikala

bigint

🔒

1	3
---	---

Tab2:

Query
Query History
AI Assistant

1
2

INSERT INTO artikli (naziv, kolicina, cena) VALUES ('Tastatura', 50, 45.00);
COMMIT;

Data Output
Messages
Notifications

WARNING: there is no transaction in progress

COMMIT

Query returned successfully in 65 msec.

Tab1:

Query
Query History
AI Assistant

1

SELECT COUNT(*) AS broj_artikala FROM artikli;

Data Output
Messages
Notifications

≡+

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

broj_artikala

bigint

🔒

1	4
---	---

Fantomsko čitanje je fenomen koji se javlja kada isti upit unutar jedne transakcije vraća različit broj redova pri uzastopnim izvršavanjima, zbog toga što je druga transakcija ubacila ili obrisala redove koji odgovaraju uslovu upita.

U demonstriranom testu, Tab 1 je pokrenuo transakciju na Read Committed nivou i prebrojao artikle u tabeli — rezultat je bio 3. Dok je transakcija bila otvorena, Tab 2 je ubacio novi artikal (Tastatura) i potvrdio promenu. Kada je Tab 1 ponovo izvršio isti COUNT upit unutar iste transakcije, rezultat je bio 4.

Novi red se "pojavió" unutar aktivne transakcije kao fantom — otuda naziv ovog fenomena. Za razliku od neponovljivog čitanja koje se odnosi na izmenu postojećih redova, fantomsko čitanje se odnosi na pojavu potpuno novih redova u skupu rezultata. Oba fenomena su moguća na Read Committed nivou izolacije.

8.8 Repeatable Read sprečava Phantom Read:

Reset:

QueryQuery HistoryAI Assistant

1

COMMIT;

2

DELETE FROM artikli WHERE naziv = 'Tastatura';

Data Output

Messages

Notifications

DELETE 1

Query returned successfully in 66 msec.

Tab1:

QueryQuery HistoryAI Assistant

1

BEGIN ISOLATION LEVEL REPEATABLE READ;

2

SELECT COUNT(*) AS broj_artikala FROM artikli;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

broj_artikala

bigint

1

3

Tab2:

Query Query History AI Assistant

1

2

INSERT INTO artikli (naziv, kolicina, cena) VALUES ('Slusalice', 30, 80.00);
COMMIT;

Data Output Messages Notifications

WARNING: there is no transaction in progress
COMMIT

Query returned successfully in 56 msec.

Tab1:

Query Query History AI Assistant

1

SELECT COUNT(*) AS broj_artikala FROM artikli;

Data Output Messages Notifications

≡+

▼

▼

SQL

broj_artikala

bigint

1	3
---	---

Kao što je pokazano u prethodnom testu, fantomsko čitanje je moguće na Read Committed nivou. Međutim, PostgreSQL ima važnu specifičnost u odnosu na SQL standard — njegov Repeatable Read nivo izolacije sprečava i fantomska čitanja, što standard pripisuje isključivo Serializable nivou.

U demonstriranom testu, Tab 1 je pokrenuo transakciju sa Repeatable Read nivoom i prebrojao artikle — rezultat je bio 3. Tab 2 je zatim ubacio novi artikal (Slušalice) i potvrdio promenu. Kada je Tab 1 ponovo izvršio isti COUNT upit, rezultat je ostao 3 — novi red bio je potpuno nevidljiv.

Ovo je direktna posledica MVCC arhitekture PostgreSQL-a. Na Repeatable Read nivou, transakcija dobija konzistentnu snimku celokupnog stanja baze u trenutku svog početka i dosledno je koristi tokom čitavog trajanja, bez obzira na ubacivanje ili brisanje redova od strane drugih transakcija. Ovo PostgreSQL čini strožijim od SQL standarda i pogodnijim za scenarije koji zahtevaju visok stepen konzistentnosti.

8.9 SELECT FOR UPDATE:

Tab1:

Query Query History AI Assistant			
1	BEGIN;		
2	SELECT id, vlasnik, stanje FROM racuni WHERE id = 1 FOR UPDATE;		

Data Output Messages Notifications			
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL			
	id [PK] integer	vlasnik character varying (100)	stanje numeric (12,2)
1	1	Ana	9000.00

Tab2:

Query Query History AI Assistant			
1	BEGIN;		
2	UPDATE racuni SET stanje = stanje - 100 WHERE id = 1;		

Data Output Messages Notifications			
Waiting for the query to complete... No data output. Execute a query to get output.			

Tab1:

Query	Query History	AI Assistant
1	COMMIT;	
Data Output	Messages	Notifications
COMMIT		
Query returned successfully in 88 msec.		

Tab2:

Query	Query History	AI Assistant
1	BEGIN;	
2	UPDATE racuni SET stanje = stanje - 100 WHERE id = 1;	
Data Output	Messages	Notifications
UPDATE 1		
Query returned successfully in 54 secs 424 msec.		

Pored automatskog zaključavanja pri operacijama pisanja, PostgreSQL omogućava eksplicitno zaključavanje redova pri čitanju kroz naredbu **SELECT FOR UPDATE**. Ovo je korisno kada aplikacija želi da pročita red, izvrši određenu logiku i potom ga izmeni, a pri tom želi da spreči da druga transakcija izmeni taj red u međuvremenu.

U demonstriranom testu, Tab 1 je zaključao Anin red naredbom **SELECT FOR UPDATE** i ostavio transakciju otvorenu. Kada je Tab 2 pokušao da izvrši **UPDATE** nad istim redom, pgAdmin je prikazao poruku "Waiting for the query to complete..." — upit je bio blokirao i nije mogao da nastavi.

Tek kada je Tab 1 izvršio **COMMIT** i otpustio zaključak, Tab 2 se odmah odblokirao i uspešno izvršio **UPDATE**. Kao što se vidi na prikazanoj poruci, Tab 2 je čekao ukupno 54 sekunde pre nego što je mogao da nastavi. Ovo jasno ilustruje kako zaključavanje štiti integritet podataka u konkurentnim okruženjima, ali i kako može uticati na performanse sistema ako se transakcije drže otvorene predugo.

8.10 NOWAIT:

Tab1:

Query	Query History	AI Assistant
1	BEGIN;	
2	SELECT * FROM racuni WHERE id = 1 FOR UPDATE;	
Data Output	Messages	Notifications
		
		
	id [PK] integer	vlasnik character varying (100)
		stanje numeric (12,2)
1	1	Ana
		8900.00

Tab2:

Query	Query History	AI Assistant
1	BEGIN;	
2	SELECT * FROM racuni WHERE id = 1 FOR UPDATE NOWAIT;	
Data Output	Messages	Notifications
ERROR: could not obtain lock on row in relation "racuni"		
SQL state: 55P03		

U prethodnom testu videli smo da SELECT FOR UPDATE blokira drugu transakciju dok zaključak ne bude oslobođen. U nekim scenarijima ovo ponašanje nije prihvatljivo — aplikacija možda ne može da čeka i mora odmah da zna da li može dobiti zaključak.

PostgreSQL pruža opciju NOWAIT koja menja ovo ponašanje: umesto čekanja, transakcija odmah dobija grešku ako ne može zaključati traženi red.

U demonstriranom testu, Tab 1 je zaključao Anin red sa FOR UPDATE i ostavio transakciju otvorenu. Kada je Tab 2 pokušao isto sa FOR UPDATE NOWAIT, PostgreSQL je trenutno vratio grešku: "could not obtain lock on row in relation racuni" — bez ikakvog čekanja.

Ovo je korisno u sistemima gde je bolje odmah obavestiti korisnika da resurs nije dostupan, nego ga držati u čekanju. Aplikacija može reagovati na ovu grešku tako što će obavestiti korisnika ili pokušati alternativnu akciju.

8.11 Deadlock:

Tab1:

Query	Query History	AI Assistant
1	BEGIN;	
2	UPDATE racuni SET stanje = stanje - 100 WHERE id = 1;	

Data Output	Messages	Notifications
UPDATE 1		
Query returned successfully in 52 msec.		

Tab2:

Query	Query History	AI Assistant
1	BEGIN;	
2	UPDATE racuni SET stanje = stanje - 100 WHERE id = 2;	

Data Output	Messages	Notifications
UPDATE 1		
Query returned successfully in 55 msec.		

Tab1:

Query	Query History	AI Assistant
1	UPDATE racuni SET stanje = stanje + 100 WHERE id = 2;	
Data Output	Messages	Notifications
UPDATE 1		
Query returned successfully in 12 secs 110 msec.		

Tab2:

Query	Query History	AI Assistant
1	UPDATE racuni SET stanje = stanje + 100 WHERE id = 1;	
Data Output	Messages	Notifications
ERROR: deadlock detected		
Process 13180 waits for ShareLock on transaction 830; blocked by process 21960.		
Process 21960 waits for ShareLock on transaction 831; blocked by process 13180.		
SQL state: 40P01		
Detail: Process 13180 waits for ShareLock on transaction 830; blocked by process 21960.		
Process 21960 waits for ShareLock on transaction 831; blocked by process 13180.		
Hint: See server log for query details.		
Context: while updating tuple (0,16) in relation "racuni"		

Deadlock je situacija u kojoj dve ili više transakcija čekaju jedna na drugu da oslobode zaključke, stvarajući kružnu zavisnost iz koje nije moguće izaći bez spoljne intervencije. PostgreSQL automatski detektuje ovakve situacije i razrešava ih poništavanjem jedne od transakcija.

U demonstriranom testu kreiran je klasičan deadlock scenario: Tab 1 je zaključao red sa id=1, a Tab 2 je zaključao red sa id=2. Zatim je Tab 1 pokušao da pristupi redu id=2 (koji drži Tab 2), dok je Tab 2 istovremeno pokušao da pristupi redu id=1 (koji drži Tab 1). Nastala je cirkularna zavisnost — nijedna transakcija nije mogla napredovati.

PostgreSQL je automatski detektovao deadlock i vratio grešku u Tab 2: "deadlock detected", uz detaljan opis cirkularne zavisnosti između procesa. Tab 1 je nastavio normalno i uspešno izvršio UPDATE. Poruka greške jasno prikazuje koji procesi su bili blokirani i kojim transakcijama su čekali, što olakšava dijagnostiku problema u produkcijskom okruženju.

8.12 Dijagnostika sa pg_locks:

Tab1:

Query Query History AI Assistant

1

BEGIN;

2

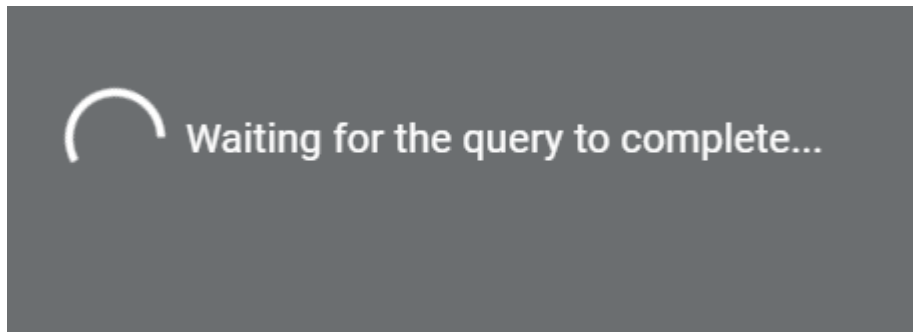
UPDATE racuni SET stanje = stanje - 100 WHERE id = 1;

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 64 msec.

Tab2:



Tab3:

Query Query History AI Assistant

1

SELECT

2

blocked.pid AS blokirani_pid,

3

blocking.pid AS blokator_pid,

4

blocked.query AS blokirani_upit,

5

blocking.query AS blokator_upit

6

FROM pg_stat_activity blocked

7

JOIN pg_stat_activity blocking

8

ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))

9

WHERE blocked.wait_event_type = 'Lock';

Data Output Messages Notifications

Showing rows: 1 to 1

	blokirani_pid integer	blokator_pid integer	blokirani_upit text	blokator_upit text
1	13180	21960	BEGIN;	BEGIN;

PostgreSQL pruža moćne alate za praćenje stanja zaključavanja u realnom vremenu kroz systemske poglede `pg_locks` i `pg_stat_activity`. Ovo je posebno korisno u produkcijskim okruženjima gde je potrebno brzo identifikovati uzrok usporavanja sistema.

U demonstriranom testu, Tab 1 je držao otvorenu transakciju sa zaključanim redom, dok je Tab 2 čekao na isti red. Upit nad `pg_stat_activity` prikazao je u realnom vremenu tačno koji proces blokira koji — proces 21960 (Tab 1) blokira proces 13180 (Tab 2). Na ovaj način administrator baze podataka može trenutno identifikovati problematičnu transakciju i po potrebi je prekinuti.

Ovo je standardna tehnika dijagnostike u PostgreSQL administraciji i nezamenljiv alat pri rešavanju problema sa performansama u okruženjima sa visokim brojem konkurentnih korisnika.

9. Zaključak

U ovom radu smo detaljno istražili fundamentalne koncepte obrade transakcija i ACID izolacije u PostgreSQL-u. Videli smo kako transakcije garantuju atomarnost i konzistentnost operacija nad podacima, i kako ACID svojstva definišu standarde koje baze podataka moraju ispuniti da bi se mogle smatrati pouzdanim.

Posebno smo se fokusirali na nivoe izolacije i fenomene koji se javljaju pri konkurentnom izvršavanju. Različiti nivoi izolacije pružaju različite kompromise između ispravnosti i performansi, a programerima je odgovornost da izaberu pravi nivo za svaki konkretan slučaj upotrebe. MVCC arhitektura PostgreSQL-a je posebno elegantno rešenje koje omogućava visoki stepen konkurentnosti bez preteranog zaključavanja — zahvaljujući MVCC-u, čitaoci i pisci mogu istovremeno raditi sa istim podacima bez međusobnog blokiranja.

Razumevanje mehanizama zaključavanja i deadlock-a je od suštinskog značaja. Disciplinovano kodiranje — konzistentni redosled zaključavanja, kratke transakcije i odgovarajuće rukovanje greškama — može efektivno sprečiti većinu problema sa zaključavanjem. PostgreSQL se ističe kao jedan od najnaprednijih sistema otvorenog koda za upravljanje bazama podataka, sa sofisticiranom implementacijom transakcija koja u nekim aspektima prevazilazi SQL standard, posebno u pogledu sprečavanja fantomskih čitanja već na nivou Repeatable Read.

Razumevanje i pravilno korišćenje transakcija i mehanizama izolacije preduslov je za izgradnju pouzdanih, sigurnih i visoko dostupnih aplikacija zasnovanih na relacionim bazama podataka. Dalji pravci istraživanja mogu uključiti distribuirane transakcije u PostgreSQL cluster okruženjima, replikaciju i njene implikacije na konzistentnost, kao i optimizaciju performansi transakcija pod visokim opterećenjem.

10. Literatura

- [1] PostgreSQL Global Development Group. PostgreSQL 16 Documentation — Chapter 13: Concurrency Control. Dostupno na: <https://www.postgresql.org/docs/current/mvcc.html>
- [2] Ramakrishnan, R. i Gehrke, J. (2003). Database Management Systems, 3. izdanje. McGraw-Hill Education. ISBN: 978-0072465631.
- [3] Stonebraker, M. i Hellerstein, J. (eds.) (2005). Readings in Database Systems, 4. izdanje. MIT Press. ISBN: 978-0262693141.
- [4] Momjian, B. (2001). PostgreSQL: Introduction and Concepts. Addison-Wesley Professional. ISBN: 978-0201703313.
- [5] Gray, J. i Reuter, A. (1992). Transaction Processing: Concepts and Techniques. Morgan Kaufmann Publishers. ISBN: 978-1558601901.
- [6] Bernstein, P.A. i Newcomer, E. (2009). Principles of Transaction Processing, 2. izdanje. Morgan Kaufmann. ISBN: 978-1558606234.
- [7] Ports, D. R. K. i Grittnr, K. (2012). Serializable Snapshot Isolation in PostgreSQL. Proceedings of the VLDB Endowment, 5(12), 1850–1861.
- [8] PostgreSQL Wiki. MVCC. Dostupno na: <https://wiki.postgresql.org/wiki/MVCC>
- [9] Obe, R. i Hsu, L. (2017). PostgreSQL Up and Running, 3. izdanje. O'Reilly Media. ISBN: 978-1491963418.
- [10] Winand, M. (2012). SQL Performance Explained. Winand & Partner. ISBN: 978-3950307825